



Universidad Nacional de Colombia

FACULTAD DE CIENCIAS

LABORATORIO 2

Advanced Data Structures
Prof. Juan Carlos Mendiavelso Moreno

Presentado por:

Angel David Gomez Pastrana

Bogotá, Julio del 2026

1. Introducción

Los árboles de búsqueda balanceados constituyen una de las estructuras de datos más utilizadas para almacenar y recuperar información de manera eficiente. Además de las operaciones fundamentales de inserción, eliminación y búsqueda puntual, en numerosas aplicaciones resulta necesario responder consultas por rango, es decir, obtener todos los elementos cuyas llaves pertenecen a un intervalo determinado. En estos escenarios, el diseño interno de la estructura tiene un impacto directo sobre el rendimiento de cada una de estas operaciones.

En este laboratorio se estudia una variante del árbol AVL en la que los registros se almacenan únicamente en los nodos hoja, mientras que los nodos internos cumplen exclusivamente la función de dirigir las búsquedas. Adicionalmente, las hojas se encuentran conectadas mediante una lista doblemente enlazada, lo que permite recorrer los elementos almacenados de forma secuencial y optimiza la ejecución de consultas por rango. Este diseño comparte principios con estructuras ampliamente utilizadas en sistemas de bases de datos, donde es indispensable mantener los datos ordenados y acceder a ellos de manera eficiente.

El objetivo principal del laboratorio es analizar el compromiso existente entre la eficiencia de las operaciones de inserción, eliminación, búsqueda puntual y búsqueda por rango. Para ello, se implementa esta variante del árbol AVL y se compara experimentalmente con un árbol AVL tradicional, evaluando el impacto que tiene el almacenamiento de los datos en las hojas y el enlazado entre estas sobre el desempeño de cada operación. De esta forma, se busca determinar si las mejoras obtenidas en las consultas por rango justifican el costo adicional que dichas modificaciones puedan introducir en las demás operaciones, identificando el balance que ofrece cada estructura según el tipo de aplicación.

2. Implementaciones

2.1. Interfaz común para los árboles

Con el objetivo de realizar una comparación justa entre las dos implementaciones desarrolladas, se definió una interfaz común denominada `AVL_Tree_Interface`. Esta interfaz establece las operaciones básicas que debe implementar cada estructura, permitiendo que ambas compartan el mismo conjunto de métodos y puedan ser utilizadas indistintamente durante las pruebas experimentales.

Esta se encuentra en `'lab2/src/interfaces/AVL_Tree_Interface'`.

Las operaciones definidas en la interfaz son las siguientes:

- `search(int key)`: determina si una llave específica se encuentra almacenada en el árbol, retornando un valor booleano.
- `insert(int key)`: inserta una nueva llave en el árbol, manteniendo las propiedades de balance correspondientes.
- `delete(int key)`: elimina una llave del árbol, realizando los ajustes necesarios para conservar la estructura balanceada.
- `rangeSearch(int a, int b)`: retorna una lista con todas las llaves cuyo valor pertenece al intervalo $[a, b]$.

- **inOrder()**: retorna una lista con todas las llaves en orden de menor a mayor.

Dado que cada árbol utiliza una representación interna diferente y sus nodos contienen información distinta, se decidió que los métodos **insert** y **delete** fueron definidos con tipo de retorno **void**, ya que para los experimentos únicamente era necesario ejecutar la operación y no obtener una referencia al nodo modificado o información adicional. De igual forma, el método **search** retorna un **boolean**, indicando únicamente si la llave se encuentra o no en el árbol, mientras que **rangeSearch** e **inOrder** retornan un objeto **List<Integer>** con todas las llaves pertenecientes al respectivo intervalo consultado.

Este diseño simplifica el tratamiento de los casos de prueba, ya que el mismo conjunto de pruebas puede ejecutarse sobre ambas implementaciones utilizando exactamente la misma interfaz. Así, el código encargado de evaluar el funcionamiento y medir el rendimiento permanece independiente de los detalles internos de cada árbol, permitiendo realizar una comparación más limpia y objetiva.

2.2. Árbol AVL Basado en Nodos

Las clases de esta implementación se encuentran en ‘lab2/src/NodeTree/’ donde su archivo es su respectivo nombre.

2.2.1. Clase Node

La clase **Node** representa la estructura básica utilizada por el árbol AVL tradicional. Cada nodo almacena la información necesaria para mantener tanto la propiedad de árbol binario de búsqueda como el balance característico de un árbol AVL.

Sus atributos son los siguientes:

- **int key**: almacena la llave asociada al nodo.
- **int height**: guarda la altura del nodo dentro del árbol. Este valor se actualiza después de cada inserción, eliminación o rotación y permite calcular el factor de balance de manera eficiente.
- **Node left**: referencia al hijo izquierdo del nodo.
- **Node right**: referencia al hijo derecho del nodo.

El constructor inicializa la llave del nodo, establece su altura en cero (al tratarse inicialmente de una hoja) y asigna referencias nulas a ambos hijos.

La clase también implementa los métodos **get** y **set** para cada uno de sus atributos. Estos métodos permiten acceder y modificar la información del nodo de forma controlada, facilitando la implementación de las operaciones del árbol sin acceder directamente a sus atributos internos.

2.3. Clase AVL_NodeTree

La implementación del árbol AVL tradicional sigue el esquema clásico de un árbol binario de búsqueda balanceado, donde cada nodo almacena una llave y referencias a sus hijos izquierdo y

derecho. Después de cada operación que modifica la estructura del árbol, se verifica el factor de balance de los nodos y, si es necesario, se realizan rotaciones para mantener la propiedad AVL.

A continuación, se describe brevemente el funcionamiento de cada uno de los métodos implementados.

- **search(int key)**
Realiza una búsqueda recursiva siguiendo la propiedad del árbol binario de búsqueda. Si la llave buscada es menor que la del nodo actual continúa por el subárbol izquierdo; si es mayor, continúa por el subárbol derecho. La búsqueda finaliza cuando encuentra la llave o alcanza un nodo nulo.
- **insert(int key)**
Inserta una nueva llave siguiendo las reglas de un árbol binario de búsqueda. Una vez realizada la inserción, durante el retorno de la recursión se actualizan las alturas y se rebalancea cada nodo visitado mediante las rotaciones correspondientes. No se permiten llaves duplicadas.
- **delete(int key)**
Localiza recursivamente el nodo que contiene la llave a eliminar. Una vez encontrado, maneja los casos clásicos de eliminación: nodo hoja, nodo con un único hijo o nodo con dos hijos, utilizando el sucesor inorden en este último caso. Finalmente, se actualizan las alturas y se rebalancean los nodos afectados.
- **rangeSearch(int a, int b)**
Recorre únicamente las ramas que pueden contener elementos dentro del intervalo $[a, b]$. Si la llave actual pertenece al rango, se agrega al resultado. Gracias a la propiedad del árbol binario de búsqueda, se evitan recorridos innecesarios sobre subárboles que no pueden contener valores válidos.
- **inOrder()**
Implementa un recorrido inOrder para el árbol AVL, así retorna una lista con todas las llaves ordenadas de menor a mayor.
- **height(Node node)**
Retorna la altura almacenada en un nodo. Si el nodo es nulo, retorna cero.
- **updateHeight(Node node)**
Recalcula la altura de un nodo como una unidad más la máxima altura entre sus hijos izquierdo y derecho.
- **balance(Node node)**
Calcula el factor de balance de un nodo como la diferencia entre la altura del subárbol izquierdo y la del derecho.
- **rebalance(Node node)**
Verifica el factor de balance del nodo e identifica cuál de los cuatro casos clásicos de desbalance (LL, LR, RR o RL) se presenta. Dependiendo del caso, ejecuta la rotación simple o doble correspondiente para restaurar el equilibrio del árbol.
- **rotateLeft(Node x)**
Realiza una rotación simple hacia la izquierda, reorganizando las referencias entre el nodo y su hijo derecho. Posteriormente actualiza las alturas de los nodos involucrados.

- **rotateRight(Node y)**
Ejecuta una rotación simple hacia la derecha utilizando el hijo izquierdo como nuevo padre del subárbol. Finalmente actualiza las alturas de los nodos modificados.
- **minValueNode(Node node)**
Obtiene el nodo con la menor llave dentro de un subárbol recorriendo sucesivamente los hijos izquierdos. Este método se utiliza para encontrar el sucesor inorden durante la operación de eliminación.

2.4. Árbol AVL Basado en Hojas

Las clases de esta implementación se encuentran en ‘lab2/src/LeafTree/’ donde su archivo es su respectivo nombre.

2.4.1. Clase abstracta Node

La clase **Node** representa la clase base para todos los nodos del árbol AVL basado en hojas. Su propósito es reunir los atributos comunes a los nodos internos y a las hojas, evitando duplicación de código.

Sus principales atributos son:

- **height**: almacena la altura del nodo, utilizada para calcular el factor de balance durante las operaciones de rebalanceo.
- **parent**: referencia al nodo padre, lo que facilita recorrer el árbol hacia arriba después de inserciones o eliminaciones.

Además, la clase define el método abstracto **isLeaf()**, el cual permite distinguir si un nodo corresponde a una hoja o a un nodo interno sin necesidad de conocer su tipo concreto.

2.4.2. Clase InternalNode

La clase **InternalNode** representa los nodos internos del árbol. A diferencia de un árbol AVL tradicional, estos nodos no almacenan registros, sino únicamente la información necesaria para dirigir las búsquedas.

Sus atributos principales son:

- **separatorKey**: llave separadora utilizada para decidir si la búsqueda continúa por el subárbol izquierdo o derecho.
- **left**: referencia al hijo izquierdo.
- **right**: referencia al hijo derecho.

Los métodos **setLeft()** y **setRight()** actualizan automáticamente el padre del nodo hijo asignado, manteniendo consistente la estructura del árbol. Finalmente, el método **isLeaf()** retorna **false**, indicando que este tipo de nodo corresponde a un nodo interno.

2.4.3. Clase LeafNode

La clase `LeafNode` representa las hojas del árbol, donde se almacenan todas las llaves del conjunto de datos.

Además de la llave, cada hoja mantiene referencias a las hojas anterior y siguiente, formando una lista doblemente enlazada ordenada.

Sus atributos principales son:

- **key**: llave almacenada en la hoja.
- **prev**: referencia a la hoja anterior.
- **next**: referencia a la hoja siguiente.

El método `linkNext()` permite enlazar una hoja con la siguiente, actualizando simultáneamente el enlace hacia atrás. Por su parte, el método `unlink()` elimina la hoja de la lista doblemente enlazada reconectando sus vecinos, operación utilizada durante la eliminación de elementos. Finalmente, `isLeaf()` retorna `true`, identificando este tipo de nodo como una hoja del árbol.

2.5. Clase AVL_LeafNode

La implementación del árbol AVL basado en hojas almacena todas las llaves exclusivamente en los nodos hoja, mientras que los nodos internos únicamente contienen llaves separadoras utilizadas para dirigir las búsquedas. Adicionalmente, todas las hojas se encuentran enlazadas mediante una lista doblemente enlazada ordenada, lo que permite recorrer secuencialmente los elementos y mejorar el desempeño de las consultas por rango.

A continuación, se describe brevemente el funcionamiento de los principales métodos implementados.

- **search(int key)**
Realiza un recorrido desde la raíz utilizando las llaves separadoras de los nodos internos hasta llegar a una hoja. Una vez alcanzada, verifica si la llave almacenada coincide con la llave buscada.
- **insert(int key)**
Localiza la hoja donde debe insertarse la nueva llave. Si la llave no existe, crea una nueva hoja, la inserta en la lista doblemente enlazada en la posición correspondiente y reemplaza la hoja original por un nuevo nodo interno que conecta ambas hojas. Finalmente, actualiza alturas, llaves separadoras y realiza el rebalanceo del árbol desde el punto de inserción hasta la raíz.
- **delete(int key)**
Busca la hoja que contiene la llave, la elimina de la lista doblemente enlazada y posteriormente elimina el nodo interno que deja de ser necesario. Finalmente, actualiza las llaves separadoras y rebalancea el árbol ascendiendo hacia la raíz.
- **inOrder()**
Busca primero la hoja mas pequeña, ubicada a la posición de mas a la izquierda del árbol,

posteriormente a través de la lista doblemente enlazada de las hojas reporta todas las llaves hasta el ultimo nodo hoja.

- **rangeSearch(int a, int b)**
Primero localiza la hoja donde comenzaría la búsqueda del valor *a*. A partir de ella, recorre la lista doblemente enlazada agregando todas las llaves hasta encontrar un valor mayor que *b*. Gracias a este recorrido secuencial, no es necesario volver a recorrer el árbol para cada elemento del intervalo.
- **findLeaf(int key)**
Recorre el árbol siguiendo las llaves separadoras hasta llegar a la hoja donde debería encontrarse la llave buscada.
- **rebalanceUp()**
Asciende desde el nodo afectado hasta la raíz actualizando alturas, llaves separadoras y aplicando las rotaciones necesarias para conservar el balance del árbol.
- **removeInternalNode()**
Después de eliminar una hoja, elimina el nodo interno que queda con un único hijo, conectando directamente dicho hijo con el abuelo correspondiente.
- **updateHeight()**
Recalcula la altura de un nodo interno a partir de las alturas de sus hijos. Las hojas mantienen altura cero.
- **balanceFactor()**
Calcula el factor de balance de un nodo como la diferencia entre las alturas de sus subárboles izquierdo y derecho.
- **updateSeparator()**
Actualiza la llave separadora de un nodo interno utilizando la menor llave almacenada en el subárbol derecho.
- **findMinimumLeaf()**
Obtiene la hoja con la menor llave dentro de un subárbol descendiendo siempre por los hijos izquierdos.
- **rotateLeft() y rotateRight()**
Realizan las rotaciones simples necesarias para restaurar el balance del árbol. Además de modificar la estructura, actualizan las alturas, las llaves separadoras y las referencias a los padres de los nodos involucrados.
- **rebalance()**
Determina si un nodo presenta alguno de los cuatro casos clásicos de desbalance (LL, LR, RR o RL) y aplica la rotación simple o doble correspondiente.
- **replaceChild()**
Sustituye uno de los hijos de un nodo interno por otro, manteniendo correctamente las referencias de la estructura.
- **insertBefore() y insertAfter()**
Insertan una nueva hoja antes o después de otra dentro de la lista doblemente enlazada, actualizando los enlaces **prev** y **next** de los nodos afectados.

3. Análisis de Complejidad

3.1. Complejidad teórica de las operaciones

Tanto el árbol AVL tradicional como el árbol AVL basado en hojas mantienen la propiedad de balance AVL, por lo que la altura del árbol permanece acotada por $\mathcal{O}(\log n)$. En consecuencia, las operaciones que requieren recorrer un camino desde la raíz hasta una hoja presentan la misma complejidad asintótica en ambos casos. La principal diferencia entre ambas estructuras se encuentra en la implementación de las consultas por rango, donde el enlazado entre hojas permite evitar recorridos repetidos del árbol.

La Tabla 1 resume las complejidades esperadas de las principales operaciones.

Operación	AVL Tradicional		AVL por Hojas	
	Promedio	Peor	Promedio	Peor
Search	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Insert	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Delete	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Range Search	$\mathcal{O}(\log n + k)$	$\mathcal{O}(n)$	$\mathcal{O}(\log n + k)$	$\mathcal{O}(n)$

Tabla 1: Complejidad teórica de las operaciones en ambas implementaciones.

A continuación se justifica brevemente la complejidad de cada operación.

3.1.1. Búsqueda puntual

En ambas implementaciones, una búsqueda consiste en descender desde la raíz hasta una hoja siguiendo la propiedad de árbol binario de búsqueda. Debido a que el árbol permanece balanceado, la altura es $\mathcal{O}(\log n)$, por lo que tanto el caso promedio como el peor caso requieren recorrer un único camino de longitud logarítmica.

3.1.2. Inserción

La inserción localiza primero la posición donde debe incorporarse la nueva llave, operación que requiere recorrer un camino de altura $\mathcal{O}(\log n)$. Posteriormente se actualizan alturas y, si es necesario, se realizan rotaciones durante el regreso hacia la raíz. Dado que únicamente se modifica el camino de búsqueda y que las rotaciones tienen costo constante, tanto el caso promedio como el peor caso presentan complejidad $\mathcal{O}(\log n)$.

En el árbol basado en hojas también es necesario insertar la nueva hoja dentro de la lista doblemente enlazada; sin embargo, esta operación se realiza en tiempo constante, por lo que no modifica la complejidad total.

3.1.3. Eliminación

La eliminación requiere localizar inicialmente la llave a eliminar, reemplazarla cuando corresponde y posteriormente actualizar el balance del árbol hasta la raíz. En ambas implementaciones

únicamente se recorren nodos pertenecientes al camino de búsqueda, mientras que las rotaciones continúan teniendo costo constante. Por ello, tanto el caso promedio como el peor caso presentan complejidad $\mathcal{O}(\log n)$.

En el árbol basado en hojas también se elimina la hoja de la lista doblemente enlazada mediante una actualización de referencias, operación cuyo costo es constante.

3.1.4. Consulta por rango

Sea k el número de llaves reportadas por la consulta.

En el árbol AVL tradicional, la búsqueda comienza localizando el inicio del intervalo y posteriormente recorre únicamente los subárboles que pueden contener elementos pertenecientes al rango consultado. Esto produce una complejidad promedio de $\mathcal{O}(\log n + k)$, ya que primero se invierte un costo logarítmico para alcanzar el intervalo y luego un costo proporcional al número de elementos reportados. En el peor caso, cuando el intervalo abarca todas las llaves almacenadas, la complejidad se convierte en $\mathcal{O}(n)$.

En el árbol AVL basado en hojas, la primera hoja del intervalo también se localiza en tiempo $\mathcal{O}(\log n)$. Sin embargo, una vez encontrada, el resto de las llaves se obtienen recorriendo la lista doblemente enlazada entre hojas, evitando volver a descender por el árbol para visitar cada elemento. Como consecuencia, su complejidad promedio también es $\mathcal{O}(\log n + k)$ y el peor caso continúa siendo $\mathcal{O}(n)$, aunque con una constante significativamente menor debido al recorrido secuencial de las hojas.

4. Planteamiento del experimento

Para iniciar, es importante mencionar algunas consideraciones sobre el desarrollo del laboratorio.

Con el fin de realizar una comparación objetiva entre ambas estructuras, primero recordemos que se implementaron dos variantes de árboles AVL: un árbol AVL tradicional y un árbol AVL basado en hojas con una lista doblemente enlazada entre sus nodos hoja. Ambas implementaciones comparten la misma interfaz y ofrecen las mismas operaciones, lo que permite que las diferencias observadas durante los experimentos se deban únicamente a sus características estructurales y no a cambios en la metodología de evaluación.

El propósito del experimento es analizar el comportamiento de ambas estructuras frente a las operaciones de inserción, eliminación, búsqueda puntual y búsqueda por rango. Para ello, se medirán los tiempos de ejecución de cada operación bajo distintos tamaños de entrada, permitiendo evaluar el balance entre el costo de mantener la estructura y la eficiencia obtenida en las consultas, especialmente en las búsquedas por rango.

4.1. Diseño del experimento

Para garantizar una comparación objetiva, ambas estructuras se evaluarán utilizando exactamente los mismos conjuntos de datos, generados con la misma semilla aleatoria para asegurar la reproducibilidad de los experimentos.

Los conjuntos de datos estarán compuestos por $n = 10\,000$, $50\,000$, $100\,000$ y $500\,000$ llaves

enteras distintas, seleccionadas uniformemente al azar del intervalo $[1, 100n]$. Sobre cada uno de estos conjuntos se desarrollarán cuatro experimentos independientes. El primero consistirá en realizar búsquedas puntuales aleatorias, midiendo el tiempo de ejecución y el promedio de nodos visitados. El segundo evaluará inserciones aleatorias, registrando el tiempo de ejecución y la cantidad de rotaciones necesarias para mantener el balance del árbol. El tercer experimento analizará consultas por rango de diferentes tamaños esperados, midiendo el tiempo de ejecución, el promedio de nodos visitados y el número de llaves reportadas. Finalmente, el cuarto experimento medirá el tiempo requerido para recorrer y reportar todos los elementos almacenados en orden ascendente. Los resultados obtenidos en cada caso se presentarán mediante gráficas en función del tamaño de entrada, permitiendo comparar el desempeño de ambas implementaciones para cada tipo de operación.

4.2. Implementación del experimento

4.2.1. Métricas

Con el objetivo de registrar las métricas necesarias durante los experimentos, a cada implementación del árbol se le incorporó un objeto de la clase `Metrics`. Esta clase mantiene contadores para el número de nodos visitados (`nodesVisited`), la cantidad de rotaciones realizadas (`rotations`) y el número de llaves reportadas en las consultas por rango (`reportedKeys`).

Para actualizar estos valores, la clase implementa los métodos `incrementNodesVisited()`, `incrementRotations()` e `incrementReportedKeys()`, los cuales son invocados en los puntos correspondientes de cada algoritmo. Además se agregaron los siguientes métodos en la interfaz para que ambas implementaciones incluyan `resetMetrics()` y `getMetrics()`, utilizados para reiniciar y recuperar las métricas de cada ejecución, respectivamente. También se añadió el método `getName()`, empleado por la clase de experimentación para identificar automáticamente la estructura evaluada.

Cada experimento utiliza únicamente las métricas que le corresponden: las búsquedas puntuales registran los nodos visitados, las inserciones contabilizan las rotaciones, las consultas por rango miden tanto los nodos visitados como las llaves reportadas y, finalmente, el recorrido secuencial únicamente requiere medir el tiempo de ejecución.

La clase `Metrics` se encuentra en `'lab2/src/Utils/Metrics'`.

4.2.2. Generación de datos

Con el fin de garantizar que ambas estructuras fueran evaluadas bajo las mismas condiciones, se desarrolló la clase `DatasetGenerator`, encargada de generar todos los conjuntos de datos utilizados durante los experimentos. Para asegurar la reproducibilidad de los resultados, se emplea una semilla fija (`SEED = 42`), de manera que todas las ejecuciones producen exactamente los mismos datos de entrada.

Esta clase implementa métodos para generar los conjuntos iniciales de llaves, búsquedas exitosas y fallidas, inserciones de nuevas llaves y consultas por rango con un tamaño esperado de salida determinado. Asimismo, incorpora el método `generateMixedSearches()`, utilizado para construir cargas de trabajo con una proporción configurable de búsquedas exitosas y fallidas. Finalmente, el método `resetSeed()` reinicia el generador de números aleatorios, permitiendo repetir los experimentos sobre los mismos conjuntos de datos cuando sea necesario.

La clase `DatasetGenerator` se encuentra en `'lab2/src/ExperimentUtils/DatasetGenerator'`.

4.2.3. Representación de resultados

Para almacenar los resultados obtenidos en cada ejecución se implementó la clase `ExperimentResult`. Esta clase encapsula toda la información producida por un experimento, incluyendo el nombre de la estructura evaluada, el tamaño del conjunto de datos, el tiempo de ejecución, el promedio de nodos visitados, el promedio de llaves reportadas y el número total de rotaciones realizadas.

De esta manera, todos los experimentos generan un objeto con el mismo formato de salida, facilitando posteriormente la comparación entre estructuras y la construcción de las gráficas correspondientes.

La clase `ExperimentResult` se encuentra en `'lab2/src/ExperimentUtils/ExperimentResult'`.

4.2.4. Consultas por rango

Con el propósito de representar de forma sencilla cada consulta por rango, se definió la clase `RangeQuery`, la cual almacena los extremos inferior y superior del intervalo, representados por las variables `a` y `b`. Esta clase se utiliza durante la generación de las cargas de trabajo y permite manejar las consultas de manera independiente de las implementaciones de los árboles.

La clase `RangeQuery` se encuentra en `'lab2/src/ExperimentUtils/RangeQuery'`.

4.2.5. Clases de experimentación

Cada uno de los experimentos fue implementado mediante una clase independiente (`SearchExperiment`, `InsertExperiment`, `RangeExperiment` y `TraversalExperiment`), todas siguiendo una misma metodología de ejecución. Para cada operación se reinician las métricas mediante `resetMetrics()`, se registra el tiempo de inicio utilizando `System.nanoTime()`, se ejecuta la operación correspondiente sobre la estructura y, finalmente, se recuperan las métricas mediante `getMetrics()`.

Cada clase acumula únicamente las métricas relevantes para el experimento que representa. En las búsquedas puntuales se registra el tiempo de ejecución y el promedio de nodos visitados; en las inserciones, el tiempo y el número total de rotaciones; en las consultas por rango, el tiempo, el promedio de nodos visitados y el promedio de llaves reportadas; mientras que el recorrido secuencial únicamente mide el tiempo requerido para recorrer todos los elementos del árbol. Al finalizar la ejecución, cada experimento construye y retorna un objeto `ExperimentResult` con los resultados obtenidos, permitiendo tratarlos de manera uniforme en las etapas posteriores del análisis.

Las clases de cada experimento se encuentran en `'lab2/src/ExperimentUtils/<NombreExperimento>'`.

5. Análisis de resultados

5.1. Experimento 1: Búsquedas Puntuales

A continuación se presentan los resultados obtenidos para el experimento de búsquedas puntuales. La Figura 1 compara ambas implementaciones en términos del tiempo total de ejecución y el número promedio de nodos visitados para los diferentes tamaños de entrada.

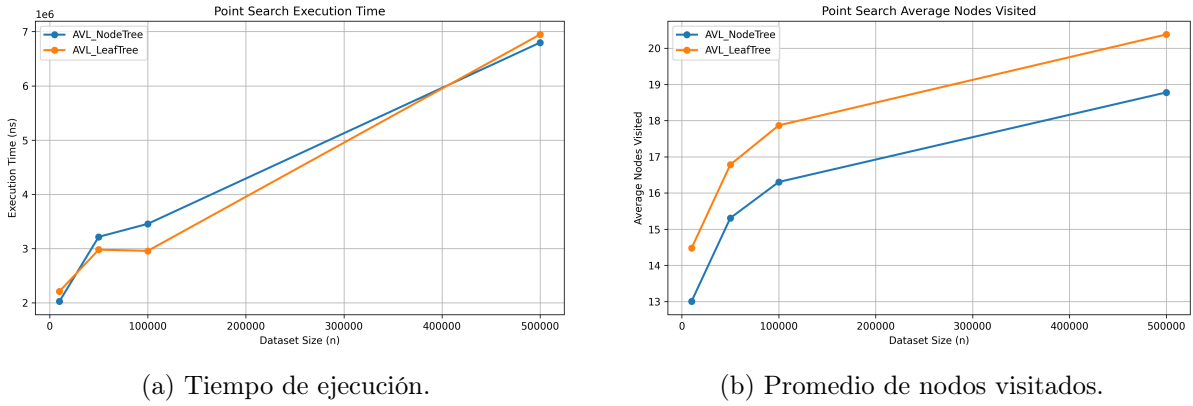


Figura 1: Resultados del Experimento 1: Búsquedas puntuales.

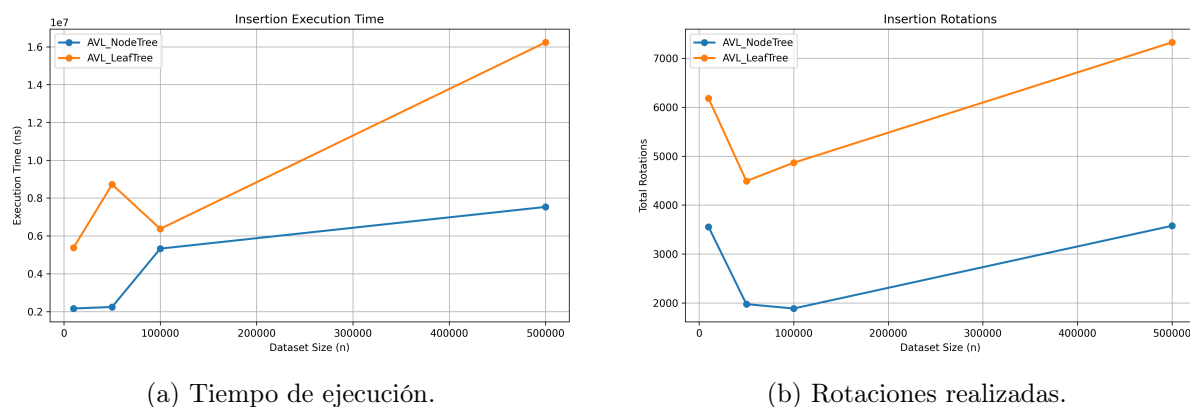
El número promedio de nodos visitados presenta un crecimiento logarítmico en ambas implementaciones, comportamiento consistente con la complejidad teórica de la búsqueda en un árbol AVL, $\mathcal{O}(\log n)$. El árbol AVL basado en hojas visita sistemáticamente entre uno y dos nodos más que el AVL tradicional, debido a que todas las llaves se almacenan en las hojas y la búsqueda siempre debe descender hasta ellas. Sin embargo, esta diferencia permanece prácticamente constante para todos los tamaños evaluados.

En cuanto al tiempo de ejecución, ambas estructuras presentan un comportamiento muy similar. Para tamaños pequeños, el AVL tradicional obtiene tiempos ligeramente menores, mientras que para tamaños intermedios el AVL basado en hojas resulta más rápido. Estas diferencias pueden atribuirse a factores de implementación, organización de memoria y optimizaciones de la JVM, cuyos efectos son comparables al costo adicional de recorrer algunos nodos más en el árbol basado en hojas. Sin embargo, estas diferencias son muy pequeñas, dada la máxima diferencia de alrededor de 0,001 segundos, por lo cual el cambio podríamos decir que no es significativo.

Para el conjunto de mayor tamaño, el AVL tradicional vuelve a presentar un tiempo ligeramente inferior; sin embargo, la diferencia es cercana al 2%, por lo que no puede considerarse una ventaja significativa. En general, ambas implementaciones ofrecen un desempeño prácticamente equivalente para búsquedas puntuales, confirmando el comportamiento logarítmico esperado y mostrando que el costo adicional del árbol basado en hojas tiene un impacto reducido sobre esta operación.

5.2. Experimento 2: Inserciones

A continuación se presentan los resultados obtenidos para el experimento de inserciones. La Figura 2 compara ambas implementaciones considerando el tiempo total de ejecución y el número de rotaciones realizadas durante la inserción de nuevas llaves.



(a) Tiempo de ejecución.

(b) Rotaciones realizadas.

Figura 2: Resultados del Experimento 2: Inserciones.

El árbol AVL tradicional realiza consistentemente un menor número de rotaciones que el árbol AVL basado en hojas. Esto se debe a que, además del rebalanceo propio de un árbol AVL, la implementación basada en hojas puede requerir modificaciones adicionales en los nodos internos para mantener la estructura de separación entre hojas. Asimismo, el número de rotaciones no presenta un crecimiento estrictamente creciente con el tamaño de la entrada, ya que depende de la secuencia específica de llaves insertadas y del estado del árbol antes de cada inserción. No obstante, en todos los conjuntos evaluados el árbol AVL basado en hojas realiza un número considerablemente mayor de rotaciones, lo que evidencia un mayor costo asociado al mantenimiento de su estructura.

El tiempo de ejecución refleja el comportamiento observado en el número de rotaciones. En todos los tamaños evaluados el árbol AVL tradicional supera al árbol basado en hojas, llegando incluso a duplicar su rendimiento para los conjuntos más grandes. Aunque ambas implementaciones conservan una complejidad teórica de $\mathcal{O}(\log n)$ por inserción, el costo constante asociado a las operaciones adicionales del árbol basado en hojas impacta directamente su tiempo de ejecución.

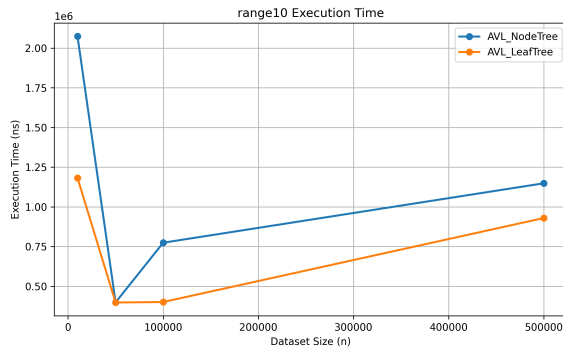
En conjunto, los resultados muestran que la inserción es una operación más costosa en el árbol AVL basado en hojas. Si bien ambas estructuras mantienen la misma complejidad asintótica, el mantenimiento de la organización de las hojas enlazadas y de los nodos internos introduce un mayor número de rotaciones y operaciones auxiliares. En consecuencia, para cargas de trabajo dominadas por inserciones, el árbol AVL tradicional constituye una alternativa más eficiente.

5.3. Experimento 3: Búsqueda por Rango

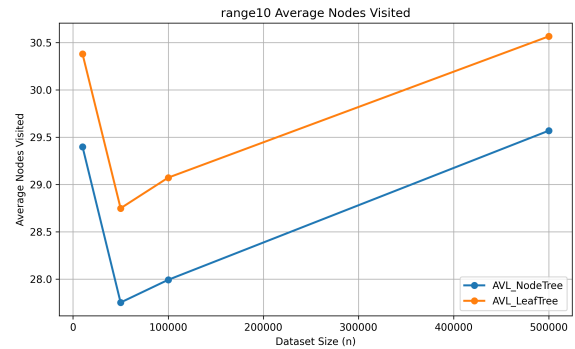
A continuación se presentan los resultados obtenidos para el experimento de consultas por rango. Se evaluaron intervalos cuyo tamaño esperado era de aproximadamente 10, 100, 1000 y 5000 llaves reportadas. Para cada caso se comparó el tiempo de ejecución y el número promedio de nodos visitados en ambas implementaciones. Los gráficos correspondientes al número promedio de llaves reportadas no se incluyen en esta sección, ya que ambas estructuras produjeron prácticamente los mismos resultados en todos los experimentos, debido a la forma en que fueron generadas las consultas por rango; estas figuras pueden consultarse en `'lab2/src/plots/images/range<número>_reported.png'`.

Y los datos en `'lab2/src/plots/csv/range<número>.csv'`.

5.3.1. Aproximadamente 10 llaves



(a) Tiempo de ejecución.

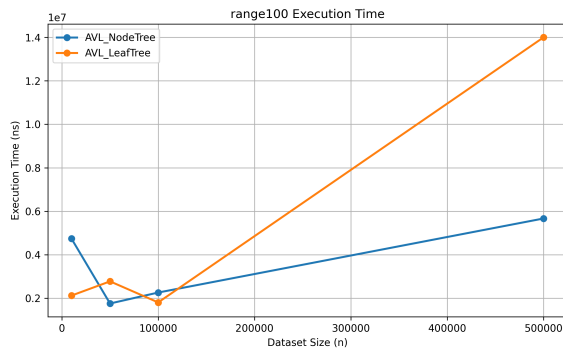


(b) Promedio de nodos visitados.

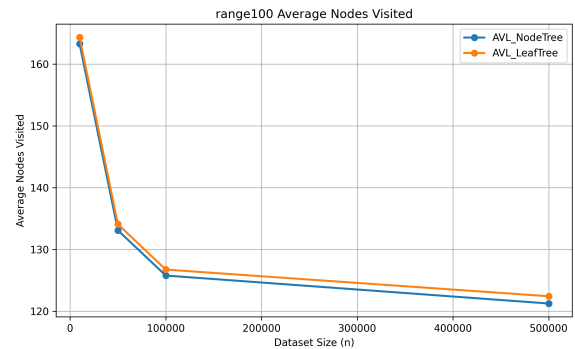
Figura 3: Resultados del Experimento 3: Búsquedas por Rango (10).

Para consultas pequeñas, ambas implementaciones presentan un comportamiento muy similar. El árbol AVL basado en hojas visita aproximadamente un nodo adicional debido a que la búsqueda siempre desciende hasta una hoja antes de iniciar el recorrido mediante los enlaces entre hojas. Sin embargo, este costo adicional es prácticamente constante y no afecta de forma significativa el tiempo de ejecución, donde ambas estructuras presentan resultados comparables.

5.3.2. Aproximadamente 100 llaves



(a) Tiempo de ejecución.

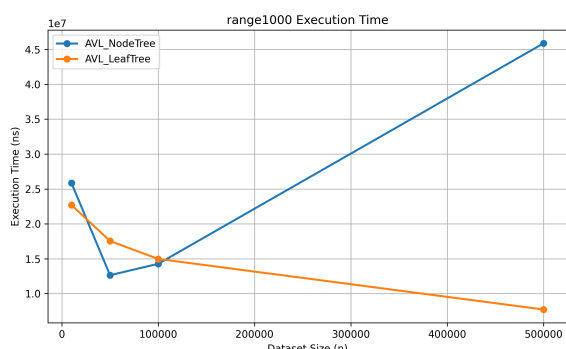


(b) Promedio de nodos visitados.

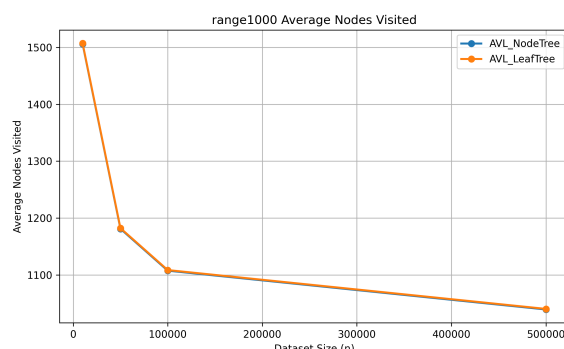
Figura 4: Resultados del Experimento 3: Búsquedas por Rango (100).

Al aumentar el tamaño del intervalo, el número de nodos visitados crece de forma proporcional al número de llaves reportadas, en concordancia con la complejidad $\mathcal{O}(\log n + k)$. En términos de tiempo, ambas implementaciones continúan mostrando un comportamiento similar, aunque aparecen pequeñas variaciones entre tamaños de entrada atribuibles a factores de implementación y ejecución de la JVM.

5.3.3. Aproximadamente 1000 llaves



(a) Tiempo de ejecución.

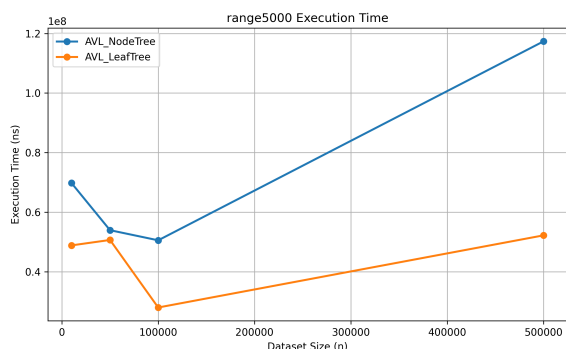


(b) Promedio de nodos visitados.

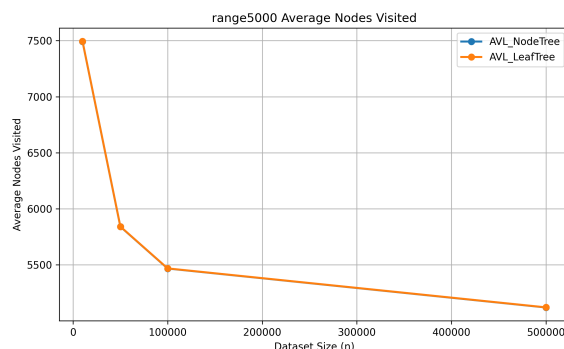
Figura 5: Resultados del Experimento 3: Búsquedas por Rango (1000).

Cuando el número de llaves reportadas es cercano a 1000, el costo asociado al recorrido del intervalo comienza a dominar sobre la búsqueda inicial. Aunque el árbol AVL basado en hojas sigue visitando ligeramente más nodos, la diferencia permanece prácticamente constante. En este escenario ya se observan ventajas en tiempo para algunos tamaños de entrada, consecuencia del recorrido secuencial mediante los enlaces entre hojas.

5.3.4. Aproximadamente 5000 llaves



(a) Tiempo de ejecución.



(b) Promedio de nodos visitados.

Figura 6: Resultados del Experimento 3: Búsquedas por Rango (5000).

Para los intervalos de mayor tamaño la ventaja del árbol AVL basado en hojas resulta más evidente. Una vez localizada la primera hoja perteneciente al rango, el recorrido continúa siguiendo los enlaces entre hojas, evitando realizar nuevas exploraciones recursivas sobre el árbol. Como consecuencia, el tiempo de ejecución tiende a ser menor que en el árbol AVL tradicional, especialmente para conjuntos de datos grandes.

5.3.5. En general

En conjunto, los resultados muestran que ambas implementaciones presentan el comportamiento esperado de una consulta por rango con complejidad $\mathcal{O}(\log n + k)$. El árbol AVL basado en hojas visita sistemáticamente alrededor de un nodo adicional debido a su estructura, diferencia que permanece prácticamente constante para todos los experimentos. Sin embargo, conforme aumenta el número de llaves reportadas, el recorrido secuencial mediante los enlaces entre hojas comienza a compensar este costo inicial, produciendo tiempos de ejecución comparables e incluso inferiores a los del árbol AVL tradicional para los intervalos más grandes. Finalmente, ambas implementaciones reportan exactamente el mismo número de llaves en todas las consultas, verificando la corrección de los algoritmos implementados.

5.4. Experimento 4: Recorrido Secuencial

A continuación se presentan los resultados obtenidos para el experimento de recorrido secuencial. La Figura 7 compara el tiempo requerido por ambas implementaciones para reportar todas las llaves almacenadas en orden ascendente.

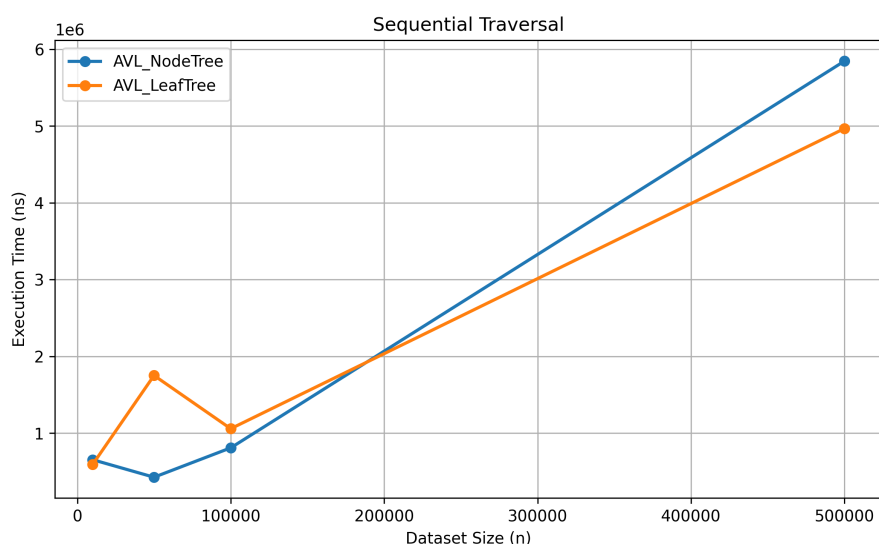


Figura 7: Resultados del Experimento 4: Recorrido Secuencial.

Ambas implementaciones presentan un crecimiento aproximadamente lineal del tiempo de ejecución, comportamiento esperado al recorrer la totalidad de las llaves almacenadas. Los resultados muestran que ninguna estructura domina consistentemente a la otra: el árbol AVL basado en hojas obtiene mejores tiempos para los conjuntos de 10 000 y 500 000 elementos, mientras que el AVL tradicional resulta ligeramente más rápido para los tamaños intermedios. En todos los casos, las diferencias corresponden a fracciones de milisegundo, por lo que ambos árboles exhiben un desempeño muy similar para esta operación.

Aunque el árbol AVL basado en hojas aprovecha los enlaces entre hojas para realizar el recorrido una vez localizada la primera de ellas, ambas implementaciones mantienen una complejidad temporal de $\mathcal{O}(n)$. Las diferencias observadas pueden atribuirse principalmente a factores de implementación y al comportamiento de la jerarquía de memoria. En estructuras de gran tamaño, la disposición de los nodos en memoria puede reducir la localidad espacial, obligando al

procesador a realizar accesos a posiciones de memoria más dispersas y aumentando el número de fallos de caché. Estos efectos introducen variaciones en los factores constantes de ejecución, lo que explica que ninguna de las dos implementaciones presente una ventaja consistente en todos los tamaños evaluados.

6. Preguntas.

6.1. ¿Porque el balanceo AVL asegura tiempo de búsqueda logarítmico?

El balanceo AVL garantiza un tiempo de búsqueda logarítmico al mantener la diferencia de alturas entre los subárboles izquierdo y derecho de cada nodo acotada por un valor máximo de uno. Esta restricción evita que el árbol se degrade hacia una estructura lineal, como podría ocurrir en un árbol binario de búsqueda sin balanceo.

Gracias a esta propiedad, la altura de un árbol AVL con n elementos permanece acotada por $\mathcal{O}(\log n)$. Dado que una operación de búsqueda consiste únicamente en recorrer un camino desde la raíz hasta el nodo buscado o hasta una hoja, el número máximo de nodos visitados es proporcional a la altura del árbol. En consecuencia, tanto el caso promedio como el peor caso de la operación `search` tienen complejidad temporal $\mathcal{O}(\log n)$.

6.2. Complejidad en el peor caso de la búsqueda por rango en AVL tradicional.

Sea n el número de llaves almacenadas y k el número de llaves reportadas en el intervalo consultado.

La búsqueda por rango consta de dos etapas: localizar el inicio del intervalo y reportar todas las llaves pertenecientes a este. La primera etapa requiere recorrer un camino desde la raíz hasta una hoja, con un costo de

$$\mathcal{O}(\log n),$$

mientras que reportar las k llaves tiene un costo de

$$\mathcal{O}(k).$$

Por tanto, la complejidad general es

$$T(n, k) = \mathcal{O}(\log n + k).$$

En el peor caso, el intervalo incluye todas las llaves del árbol, es decir, $k = n$. Sustituyendo este valor se obtiene

$$T(n, n) = \mathcal{O}(\log n + n) = \mathcal{O}(n),$$

6.3. Complejidad en el peor caso de la búsqueda por rango en el AVL basado en hojas

En el árbol AVL basado en hojas, la búsqueda por rango inicia descendiendo desde la raíz hasta localizar la primera hoja perteneciente al intervalo. Debido a que el árbol permanece balanceado, esta búsqueda tiene un costo de

$$\mathcal{O}(\log n).$$

Una vez encontrada la primera hoja, las restantes llaves del intervalo se obtienen siguiendo los enlaces de la lista doblemente enlazada entre hojas. Como cada una de las k llaves reportadas se visita exactamente una vez, esta etapa tiene un costo de

$$\mathcal{O}(k).$$

Por lo tanto, la complejidad total de la búsqueda por rango es

$$T(n, k) = \mathcal{O}(\log n + k).$$

En el peor caso, cuando el intervalo incluye todas las llaves almacenadas ($k = n$), se obtiene

$$T(n, n) = \mathcal{O}(\log n + n) = \mathcal{O}(n).$$

6.4. ¿Por qué el AVL basado en hojas puede tener un mejor desempeño en la práctica?

Aunque ambas implementaciones tienen una complejidad de $\mathcal{O}(\log n + k)$, el árbol AVL basado en hojas presenta menores factores constantes durante el recorrido del intervalo. Una vez localizada la primera hoja, las llaves restantes se obtienen siguiendo los enlaces entre hojas, evitando regresar repetidamente a los nodos internos del árbol.

Este comportamiento se refleja en los resultados experimentales: para consultas pequeñas ambos árboles presentan tiempos similares, mientras que para rangos de mayor tamaño (1000 y 5000 llaves) el AVL basado en hojas tiende a obtener mejores tiempos de ejecución, especialmente para valores grandes de n . Por tanto, aunque la complejidad asintótica es la misma, la organización de las hojas permite un recorrido más eficiente en la práctica.

6.5. ¿Qué estructura obtuvo un mejor desempeño?

- **Búsquedas puntuales:** Ambas implementaciones presentaron un desempeño muy similar. Para $n = 10\,000$ y $n = 500\,000$ el árbol AVL tradicional obtuvo menores tiempos de ejecución, mientras que para $n = 50\,000$ y $n = 100\,000$ el árbol AVL basado en hojas fue ligeramente más rápido. No obstante, las diferencias fueron del orden de fracciones de milisegundo. En cuanto al número de nodos visitados, el AVL basado en hojas recorrió sistemáticamente alrededor de un nodo adicional, ya que siempre debe descender hasta una hoja para confirmar la existencia de la llave.

- **Consultas por rango pequeñas:** Para intervalos de aproximadamente 10 y 100 llaves, ambas implementaciones presentaron tiempos de ejecución muy similares. En estos casos, el costo de localizar el inicio del intervalo domina la operación, por lo que la ventaja del recorrido mediante hojas enlazadas aún no resulta significativa.
- **Consultas por rango grandes:** Para intervalos de aproximadamente 1000 y 5000 llaves, el árbol AVL basado en hojas mostró un mejor desempeño en la mayoría de los casos, especialmente para los conjuntos de datos más grandes. Una vez localizada la primera hoja del intervalo, las llaves restantes se recorren siguiendo los enlaces entre hojas, evitando regresar repetidamente a los nodos internos como ocurre en el recorrido *in-order* del AVL tradicional. Sin embargo, cuando el recorrido abarca una porción muy grande o incluso la totalidad de la estructura, esta ventaja comienza a disminuir debido a la baja localidad espacial propia de las estructuras enlazadas. Al encontrarse las hojas distribuidas en distintas posiciones, el procesador realiza accesos a memoria menos eficientes, incrementando los fallos de caché y reduciendo parte del beneficio obtenido por el recorrido secuencial.

6.6. ¿Cómo se asemeja el árbol AVL basado en hojas a un árbol B+?

El árbol AVL basado en hojas comparte con un árbol B+ la idea de almacenar las llaves únicamente en las hojas, mientras que los nodos internos se utilizan exclusivamente para dirigir la búsqueda mediante llaves separadoras. Además, las hojas se encuentran enlazadas secuencialmente, permitiendo recorrer los elementos en orden sin necesidad de volver a recorrer la estructura interna del árbol.

Esta organización hace que las consultas por rango sean más eficientes en la práctica, ya que, una vez localizada la primera hoja del intervalo, las llaves restantes se obtienen siguiendo los enlaces entre hojas. La principal diferencia es que el árbol B+ es un árbol de múltiples caminos diseñado para optimizar el acceso a memoria secundaria, mientras que el AVL basado en hojas mantiene una estructura binaria balanceada orientada al almacenamiento en memoria principal.

7. Conclusiones

En este laboratorio se implementaron y compararon dos variantes de árboles AVL: un árbol AVL tradicional y un árbol AVL basado en hojas con una lista doblemente enlazada entre sus hojas. Ambas implementaciones preservan las propiedades de balance del árbol AVL, garantizando un tiempo de ejecución logarítmico para las operaciones de búsqueda, inserción y eliminación, así como una complejidad de $\mathcal{O}(\log n + k)$ para las consultas por rango.

Los resultados experimentales mostraron que el árbol AVL tradicional presenta un mejor desempeño en las operaciones de inserción y, en general, en las búsquedas puntuales, debido a que requiere visitar un menor número de nodos. En contraste, el árbol AVL basado en hojas obtiene ventajas en consultas por rango de gran tamaño, ya que el recorrido mediante los enlaces entre hojas reduce el trabajo necesario una vez localizado el inicio del intervalo.

No obstante, también se observó que esta ventaja práctica no crece indefinidamente. Cuando el recorrido involucra una gran cantidad de elementos, factores de implementación como la localidad espacial de memoria y el acceso mediante punteros comienzan a influir en el tiempo de ejecución, reduciendo parte del beneficio obtenido por la estructura enlazada.

En conclusión, ninguna de las dos estructuras es superior en todos los escenarios. El árbol AVL

tradicional constituye una mejor alternativa cuando predominan las operaciones de búsqueda puntual e inserción, mientras que el árbol AVL basado en hojas resulta más conveniente para aplicaciones donde las consultas por rango representan una fracción importante de la carga de trabajo. Esto evidencia que, además de la complejidad asintótica, los factores constantes y la organización interna de las estructuras de datos desempeñan un papel importante en su rendimiento práctico.